

Repaso parcial 1

Pruebas

Herramienta	Tipo de Prueba	Ventaja	Donde utilizarla
Jest	Pruebas unitarias y de integración para JavaScript	Rápido y con buena integración en proyectos React	1. Probar componentes de React en una aplicación de e-commerce. 2. Verificar funciones de backend
Mocha	Pruebas unitarias y de integración para JavaScript	Flexible y compatible con muchas bibliotecas de aserción	1. Probar funciones o módulos de una app
ZAP (OWASP ZAP)	Pruebas de seguridad	Escáner de vulnerabilidades gratuito y fácil de usar	Detectar vulnerabilidades en una tienda en línea
Jira	Gestión de pruebas y seguimiento de errores	Permite rastrear incidencias y gestionar sprints ágilmente	Organizar pruebas en un equipo de desarrollo
JUnit	Pruebas unitarias para Java	Integración con herramientas CI/CD y fácil de usar	Validar clases en una aplicación de facturación
Pytest	Pruebas unitarias y funcionales en Python	Sintaxis sencilla y soporte para múltiples tipos de prueba	Probar una función de procesamiento de datos
Katalon	Pruebas automatizadas (web, API, móviles)	Interfaz visual amigable y sin necesidad de programación avanzada	Validar formularios en una app
Pitest	Pruebas de mutación en Java	Detecta fallos en pruebas unitarias con alta precisión	Validar la efectividad de pruebas en un software financiero

Mutation Testing

Es una técnica avanzada de pruebas de software que evalúa la calidad de las pruebas unitarias modificando (mutando) pequeñas partes del código y verificando si las pruebas

existentes detectan estos cambios. Su objetivo es identificar debilidades en las pruebas y mejorar la cobertura.

- Un mutante muere, cuando un cambio en el código (mutación) provoca que las pruebas fallen, es decir, que las pruebas detectaron el cambio entonces se puede decir que son efectivas
- Un mutante sobrevive, cuando una mutación no causa fallos, esto significa que las pruebas no son lo suficientemente buenas y pueden estar dejando pasar errores.

Herramienta utilizada

MutPy

- ¿Dónde utilizar mutation testing?
 - En proyectos críticos donde la confiabilidad del código es esencial, como en software financiero, salud o sistemas de control industrial.
 - En un software financiero, para probar el módulo de cálculo de impuestos, asegurando que las pruebas unitarias detecten cualquier error en los cálculos.

Devops

Es un enfoque de desarrollo que integra *desarrollo (Dev)* y *operaciones (Ops)* para acelerar la entrega de aplicaciones, mejorar la colaboración entre equipos y aumentar la calidad del software mediante automatización y monitoreo continuo.

¿Dónde implementar DevOps?

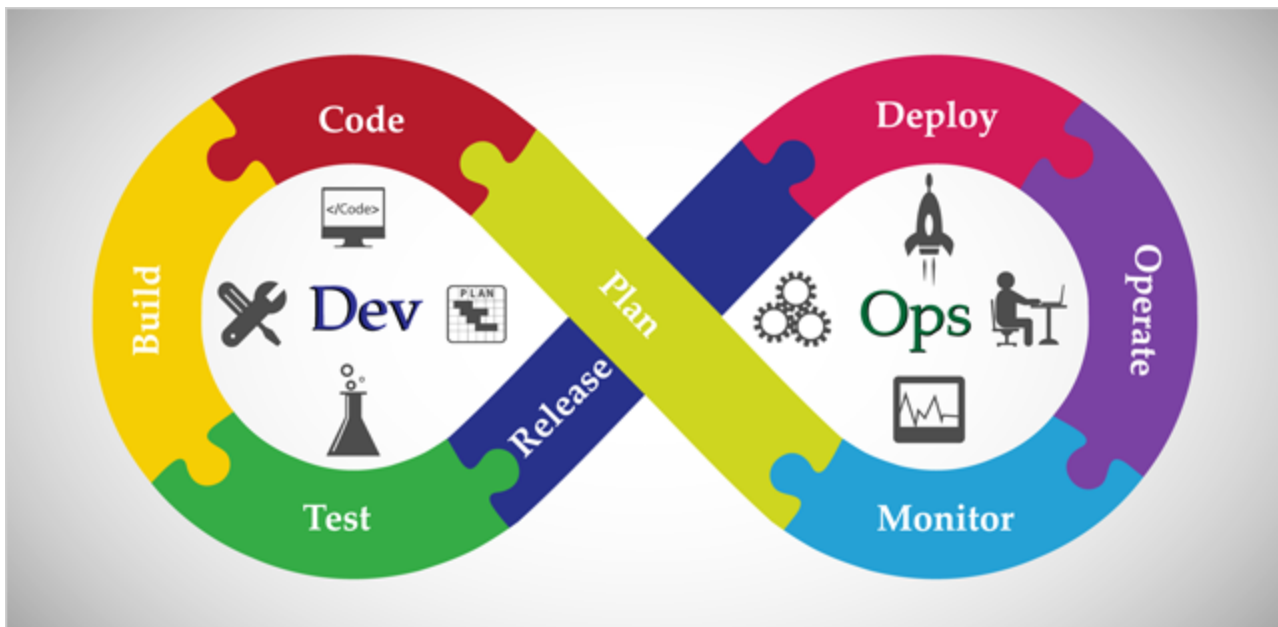
1. Empresas que necesiten despliegues frecuentes
2. Equipos que trabajen en arquitecturas en la nube
3. Organizaciones que buscan automatizar pruebas y despliegues

CI (Integración Continua)

Automatiza la integración del código en un repositorio compartido, asegurando que cada cambio pase pruebas antes de fusionarse.

CD (Despliegue Continuo/Entrega)

- Continuos Delivery, automatiza la prueba del código a un entorno listo para producción, pero con una aprobación manual para el despliegue final.
- Continuos Deployment: automatiza la entrega y despliegue sin intervenciones humanas.



Ciclo de vida de DevOps

1. Plan (planificación), definir requerimientos, tareas y estimación de tiempos. *Herramientas: Jira, Trello*
2. Code (desarrollo - codificación), los programadores empiezan a escribir el código y a versionarlo. *Herramientas: Git, GitHub*
3. Build (construcción), se junta todo el código y se genera un ejecutable. *Herramientas: Maven, Gradle*
4. Test (pruebas), se prueba para verificar que todo funciona correctamente en base a los requerimientos, las pruebas se automatizan. *Herramientas: Pytest, JUnit*
5. Release (lanzamiento), el software pasa a una versión lista para producción, pero aún no se implementa. *Herramientas: Docker, Kubernetes*
6. Deploy (despliegue), se despliega en servidores de prueba y ya está al alcance de los usuarios finales. *Herramientas: Jenkins, AWS CodeDeploy*
7. Operate (funcionar), ocurre todo el tiempo, porque básicamente son configuraciones y optimizaciones. *Herramientas: Terraform, Kubernetes*
8. Monitor, se supervisa el rendimiento de la aplicación y se detectan fallos en tiempo real. *Herramientas: Grafana, Datadog*

Métricas

Son valores numéricos que permiten medir la calidad, eficiencia y progreso del desarrollo de software. Se usan para evaluar rendimiento, esfuerzo, complejidad y calidad del código.

¿Para qué sirven?

- Medir la productividad del equipo de desarrollo

- Evaluar la calidad del código y detectar problemas tempranos
- Predecir costos y esfuerzo requerido para un proyecto
- Mejorar la planificación y optimización del desarrollo

Métricas comunes en el desarrollo de software

1. Líneas de código (LOC), cantidad de líneas de código de un proyecto
2. Defectos por KLOC, cantidad de errores por cada 1000 línea de código
3. Tiempo de respuesta, tiempo que tarda un sistema en responder una solicitud
4. Tasa de fallos, número de errores por unidad de tiempo

Punto de función

Mide la funcionalidad del software basándose en la interacción del usuario con el sistema. No mide líneas de código, sino el tamaño en funcional del sistema. *No mide el código si no lo que el software hace para el usuario*

Valor de dominio de información	Conteo	Simple	Promedio	Complejo				
Entradas externas (EE)	15		4		=B3*D3			
Salidas externas (SE)	10		5		50			
Consultas externas (CE)	12		4		48			
Archivos lógicos internos (ALI)	8		10		80			
Archivos de interfaz externos (AIE)	1		7		7			
Conteo Total					245			
PF =	232.75							
			PF = conteoTotal*(0.65*0.01*sumaMultiplicacion(conteo*promedio))					